LAB 7 — FUZZING FOR INPUT VALIDATION BUGS

Software Engineering Laboratory

Student: Tanmay Sridhar
SRN: PES1UG25CS849
Experiment: L6_L7_04 — Shipping Quotation

Aim

To identify malformed-input failures in a shipping quotation application through property-based fuzz testing, correct the input validation, and demonstrate the difference between the original and fixed implementations.

Tools and project files

Python 3, pytest and Hypothesis. The assigned starter is the shipping quotation Lab 7 repository. **buggy_processor.py** preserves the initial implementation; **processor.py** contains the corrected implementation; **test_processor.py** contains property-based and regression tests.

Functions under test

1. calculate_dimensional_weight(dimensions_str): reads three dimensions separated by x and computes (length × width × height) / 5000, rounded to two decimals.

2. format_tracking_label(raw_code): converts a tracking identifier such as E123 into E-000123.

3. parse_package_manifest(json_payload): extracts the weight value from a JSON manifest.

Method

First, execute the buggy script with malformed input. Next, run the Hypothesis/pytest suite against the original code to discover failures. Add type, format, range and finite-number validation. Finally, rerun the application and tests against the fixed code.

Recorded outcomes

The supplied execution evidence shows **21 failed, 8 passed** against the buggy code, followed by **29 passed** against the corrected code. The original script terminates with IndexError on 1x2; the corrected script reports a readable validation error and continues.

The four evidence images on the following pages are taken from the supplied PES1UG25CS849 execution record; no test outputs have been fabricated.

Evidence 1 — Buggy application

When the user enters 1x2 instead of three dimensions, the original parser attempts to read a missing third element and crashes with IndexError.

Terminal output

```
$ python3 buggy_processor.py
Input: '1x2\nE123\n{"weight":12.5}\n'
==== Running calculate_dimensional_weight ====
Enter dimensions LxWxH in cm (e.g. 20x30x10): Traceback (most recent call last):
  File "/Users/tanmaysridhar/SE-Labs--PES1UIG25CS849-/Lab 7/buggy_processor.py", line 36, in <module>
    main()
  File "/Users/tanmaysridhar/SE-Labs--PES1UIG25CS849-/Lab 7/buggy_processor.py", line 22, in main
    dim_weight = calculate_dimensional_weight(dims_input)
                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Users/tanmaysridhar/SE-Labs--PES1UIG25CS849-/Lab 7/buggy_processor.py", line 7, in calculate_dimensional_weight
    height = float(dims[2])
                   ~~~~^^^
IndexError: list index out of range

Exit code: 1
```

Figure 1. Recorded terminal evidence from the PES1UG25CS849 Lab 7 run.

Evidence 2 — Failures found by fuzzing

The initial pytest run reports 21 failures and 8 passes. The failures include missing dimensions, invalid tracking identifiers, malformed manifests, and unsupported types.

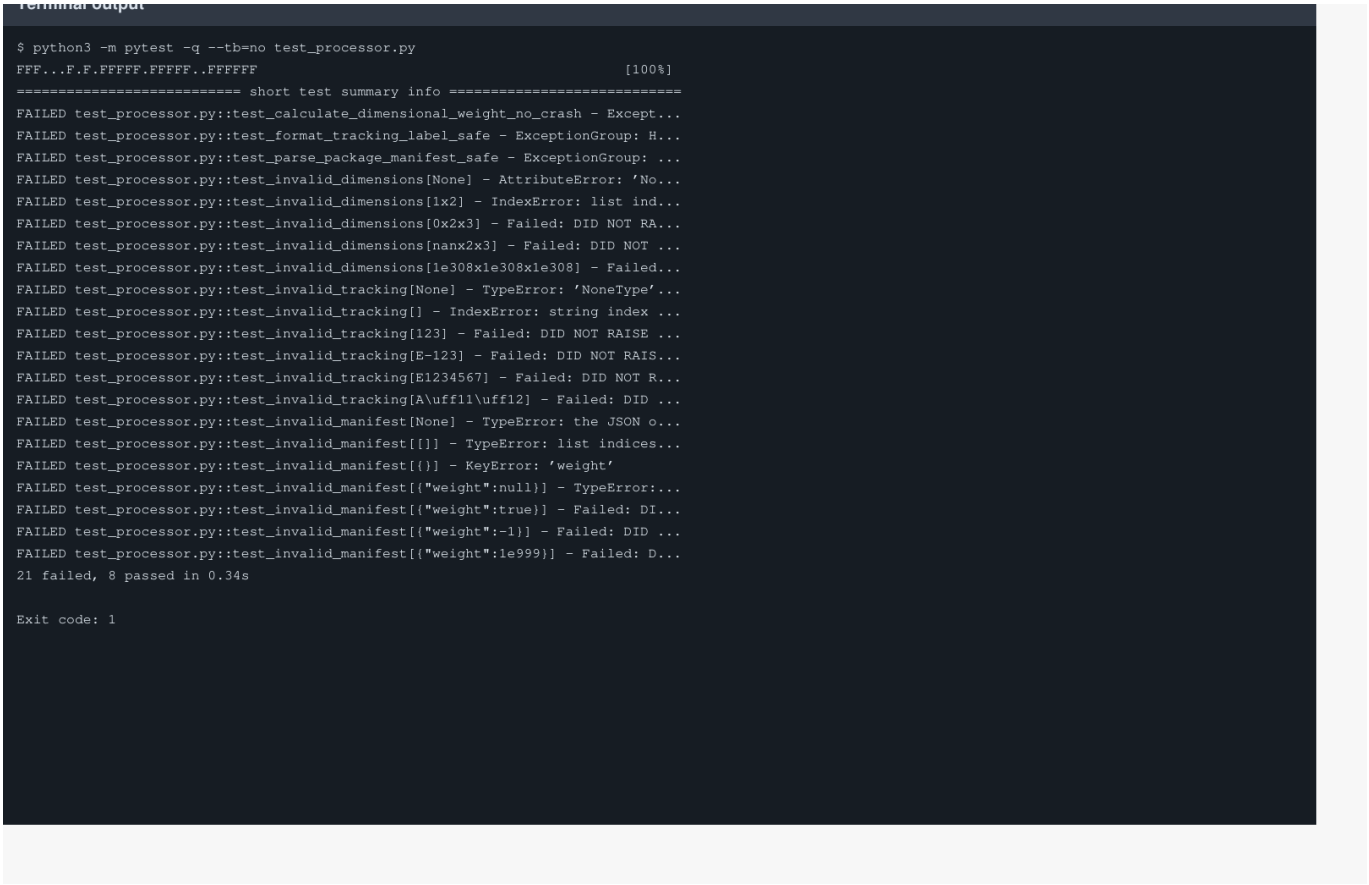


Figure 2. Recorded terminal evidence from the PES1UG25CS849 Lab 7 run.

Evidence 3 — Application after fixing validation

The corrected application rejects 1x2 with an explanatory message instead of crashing. The next inputs still work: E123 becomes E-000123 and manifest weight 12.5 is returned.

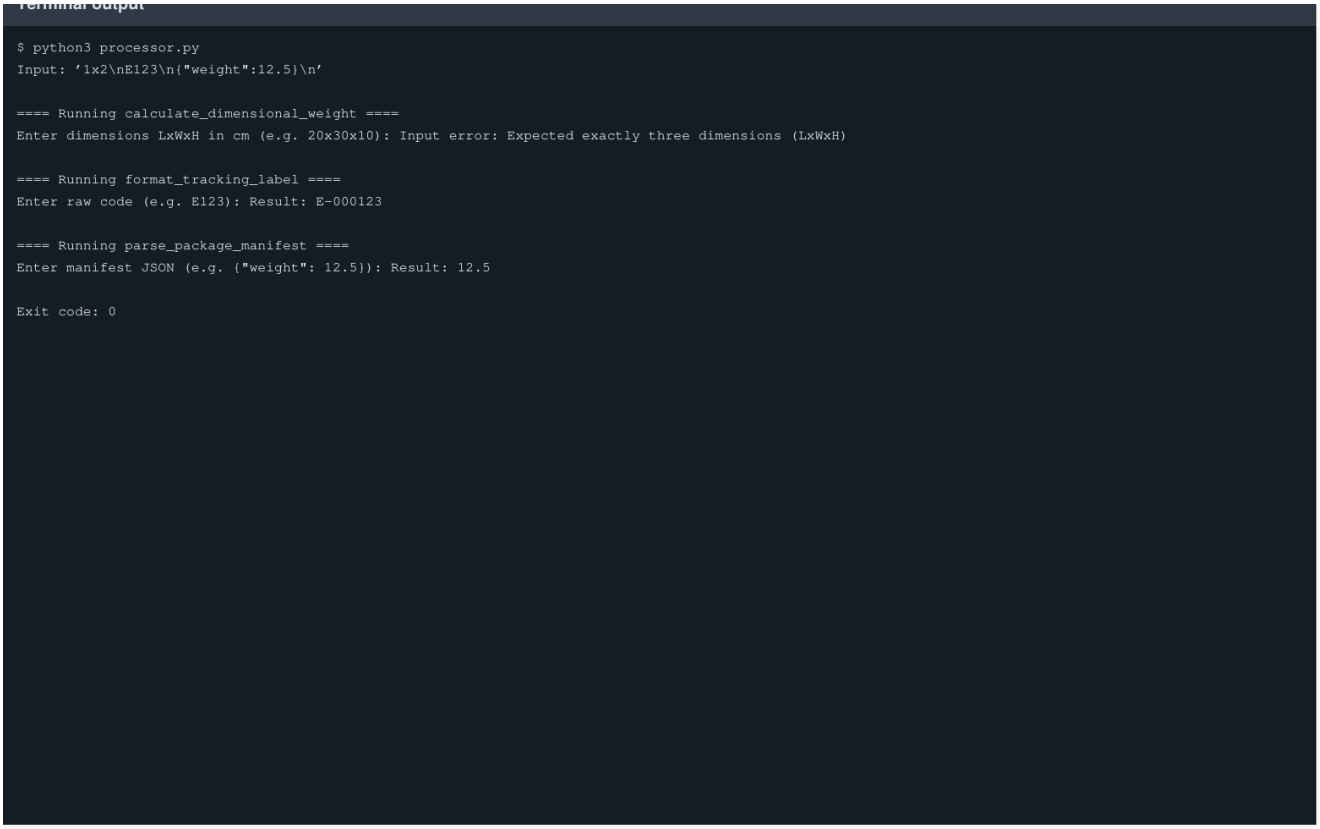


Figure 3. Recorded terminal evidence from the PES1UG25CS849 Lab 7 run.

Evidence 4 — Successful regression testing

The same test suite is rerun against the corrected implementation. The recorded output shows all 29 tests passing.

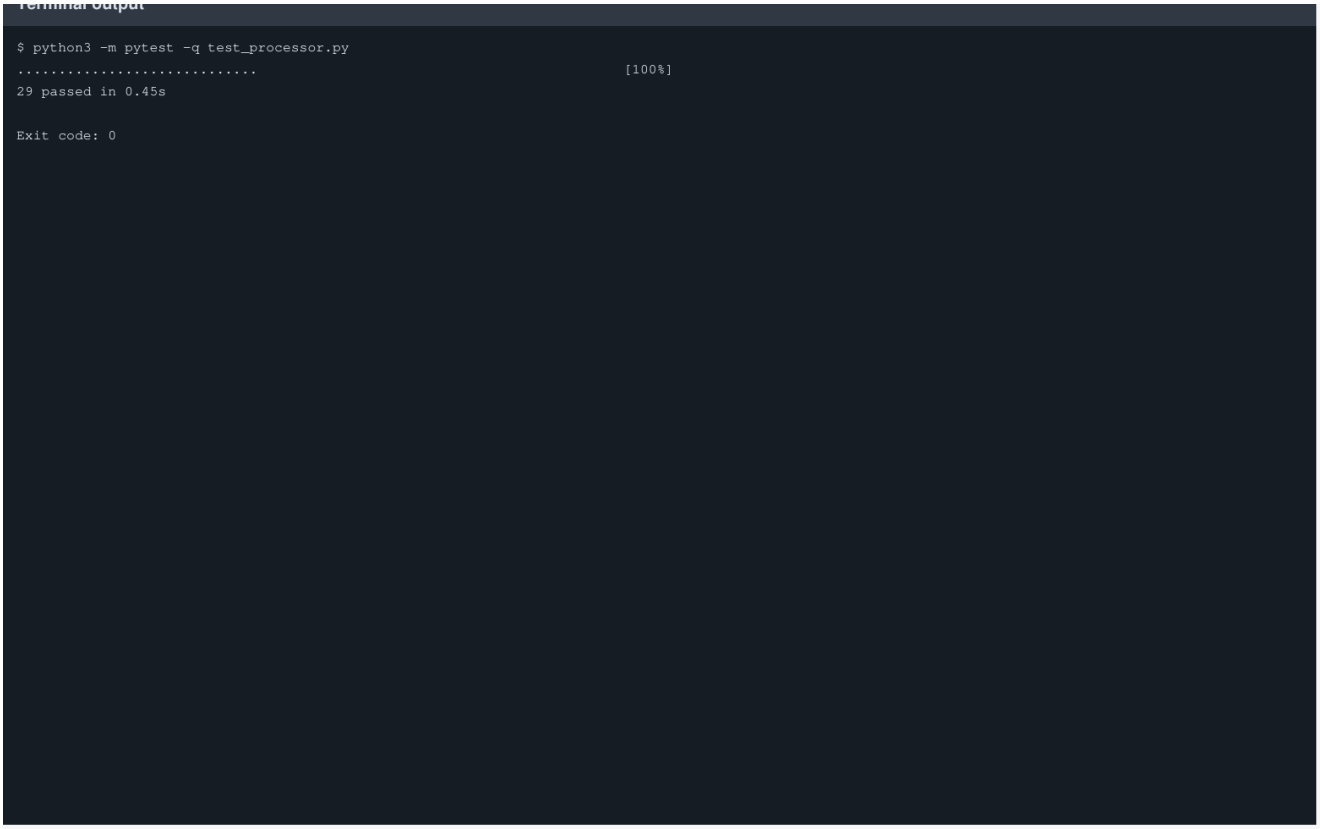


Figure 4. Recorded terminal evidence from the PES1UG25CS849 Lab 7 run.

ANALYSIS AND CONCLUSION

Bugs identified

Dimensional weight: The original implementation assumes exactly three x-separated numeric fields. Missing fields raise IndexError; invalid strings or null-like values can cause other exceptions; NaN and negative dimensions require explicit rejection.

Tracking label: The original implementation indexes the first character without checking whether the input is empty and parses the rest as an integer without verifying the code format.

Package manifest: The original implementation assumes valid JSON with a weight key. Missing keys, arrays, null values and inappropriate weight types can produce exceptions or incorrect results.

Corrections applied

The corrected functions check input types, required structure and allowed values before computing. Invalid inputs are rejected using ValueError; the interactive program handles those errors and proceeds to the next operation.

Results table

Stage	Observed result
Original application	IndexError for incomplete dimensions
Tests before fix	21 failed; 8 passed
Fixed application	Input error shown; subsequent inputs processed
Tests after fix	29 passed

Conclusion

Property-based fuzzing revealed input validation weaknesses that ordinary valid-input runs could miss. The corrected shipping quotation functions handle unexpected values more safely. The supplied test evidence shows the regression suite passing after the fixes.

Reference: Assigned Lab 7 shipping quotation repository and Lab 7 student handout.